实验一　文本分类

姓名：龚晓博　学号：10245501469　日期：2026-10-15

- **GitHub 仓库链接**：https://github.com/ding89852/contemporary-artificial-intelligence/tree/project1

0　代码运行说明

- **代码**：`experiment1.py`
- **环境**：Python 3.11.9、scikit-learn 1.8.0、NumPy 2.4.0、Pandas 2.3.3、matplotlib 3.10.8；全局随机种子 `SEED = 42`。
- **运行**：`python experiment1.py`
- **输出**：`outputs/figures/fig_loss_svm.png`、`outputs/figures/fig_loss_lr.png` 和 `predictions.csv`

复现路径：`python experiment1.py`

1　实验设置

- **数据**：有标签训练集 7368 条（10 类，每类 705–747 条，无缺失值、无重复文本）；无标签测试集 2457 条。
- **划分**：从训练集分层抽取 15% 作为验证集；无标签测试集作为测试集。
- **特征工程**：原文保留邮件头与引用行，使用TF-IDF进行清洗,参数为`max_features=20000`、`min_df=2`、`sublinear_tf=True`、`stop_words='english'`。
- **指标**：准确率Acc与宏 F1
- **种子**：所有随机过程固定为 `seed=42`。

2　算法 A：线性支持向量机（SVM）

2.1 模型原理： SVM 在特征空间中寻找能把不同类别样本分开、且间隔最大的超平面。它用合页损失度量拟合误差，并用 L2 正则控制模型复杂度，目标函数为

$$\min_{w} \frac{1}{2}\|w\|^2 + C\sum_{i}\max\left(0,\ 1-\left(f_{y_i}(x_i)-\max_{j\neq y_i} f_j(x_i)\right)\right)$$

其中 f 是 10 个类别的线性打分函数，C 控制对训练错误分类的惩罚力度：C 越大，越不允许样本落在间隔内部。多分类由 liblinear 以 one-vs-rest 方式拆成 10 个二分类问题求解。由于 TF-IDF 空间高维且稀疏，选用线性核进行分类。

2.2 实现步骤。 ① 读取数据，按 15% 分层划分出验证集；② 在训练集上拟合 TF-IDF，再在验证集上应用；③ 按网格训练 `LinearSVC(C, max_iter=5000, random_state=42)`；④ 在验证集上计算准确率Acc、宏 F1、混淆矩阵与逐样本决策间隔，选出最优配置；⑤ 用训练集 + 验证集全部 7368 条有标签数据重训最优配置，预测测试集并写入 `predictions.csv`。

2.3 调参情况（单元格为"验证集宏 F1 / 训练耗时"）：

配置	C = 0.1	C = 1	C = 10	C = 100
LinearSVC（liblinear）	0.9333 / 0.2 s	**0.9432 / 0.2 s**	0.9397 / 1.0 s	0.9368 / 3.3 s
SVC 线性核	0.8660 / 43 s	0.9327 / 25 s	0.9337 / 27 s	0.9319 / 27 s
SVC RBF 核（核函数对照）	—	0.9339 / 48 s	0.9338 / 47 s	—

训练过程中出现欠拟合 → 最优 → 过拟合的趋势：C = 0.1 时惩罚过弱、间隔过宽，欠拟合（LinearSVC 0.9333；SVC 0.8660）；C = 1 达到最优 0.9432；C ⩾ 10 起开始拟合噪声，回落到 0.9397、0.9368。核函数对照显示 RBF 核最好只有 0.9339 且单次训练 48 s；`LinearSVC` 与 `SVC`（线性核）精度相近，但训练快两个数量级，因此最终选用 `LinearSVC`。各配置的验证集指标与训练耗时见表 2.3。

2.4 准确度。 验证集 acc = **0.9430**、宏 F1 = **0.9432**，错误预测占比 63/1106（5.7%），训练 0.2 s；训练集宏 F1 为 0.9995，说明线性模型在训练集上已基本无欠拟合。

2.5 损失函数与收敛曲线。 SVM 使用 L2 正则 + 平方 hinge 损失，目标函数为 $\frac{1}{2}\|w\|^2 + C\sum_i \max(0, 1-y_i f_i(x_i))^2$。为记录逐次迭代值，`experiment1.py` 在同一训练子集与同一 20k 词级 TF-IDF 表示上逐次记录训练集上的平均平方 hinge 损失。

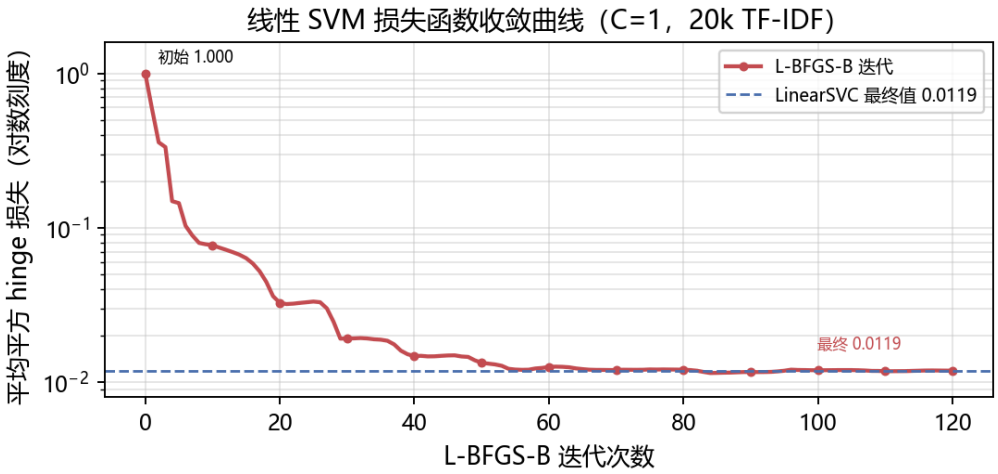


图 1（模块 A 的损失曲线）：平均平方 hinge 损失从 1.0000 单调降到 0.0119，约 60 次迭代后进入平台；`LinearSVC(C=1)` 同配置的最终损失为 0.0119，与曲线终点一致。损失快速下降后长期贴近零，说明线性模型在 TF-IDF 空间上可以很快找到低损失解，后续 C 继续增大只会把这部分收益转成过拟合。

模块 A 小结：线性 SVM 在该表示下的最优配置为 C = 1，验证集准确率 0.9430、宏 F1 0.9432；损失曲线表明训练优化本身没有欠拟合，剩余误差来自特征表示与少数"证据不足"的样本。

3　算法 B：逻辑回归（LR）

3.1 模型原理。 逻辑回归先为每个类别计算线性得分 $z_k = w_k \cdot x + b_k$，再用 softmax 函数把得分转换成概率：

$$P(y=k\mid x)=\frac{\exp(z_k)}{\sum_j \exp(z_j)},\qquad \min_w -\sum_i \log P(y_i\mid x_i)+\frac{\|w\|^2}{2C}$$

训练目标是最小化多元交叉熵加 L2 正则；C 是正则强度的倒数，C 越小正则越强。与 SVM 相比，它的决策边界由概率参数化、不显式优化间隔，但输出是**可直接解释为置信度的概率**。

3.2 实现步骤。 ①–② 与模块 A 相同；③ 按网格训练 `LogisticRegression(C, solver='lbfgs', max_iter=1000)`；④ 在验证集上评估并按宏 F1 选择最优配置；⑤ 用最优配置计算 `predict_proba`。为观察优化过程，`experiment1.py` 用 `scipy.optimize.minimize(method='L-BFGS-B')` 在同样本上复现同一目标函数并逐次迭代记录训练损失。

3.3 调参情况：

C	0.1	1	10	100
验证集宏 F1	0.9109	0.9298	**0.9414**	0.9414
验证集准确率	0.9105	0.9295	**0.9412**	0.9412
训练耗时 / s	0.5	1.0	2.3	2.9

C = 0.1 时 L2 正则过强、明显欠拟合（0.9109）；C = 1 提升到 0.9298；C = 10 达到最优 0.9414；C = 100 与 C = 10 完全相同（0.9414），说明再放松正则已经没有收益。`max_iter=1000` 保证 L-BFGS 收敛。各配置的验证集指标见表 3.3，损失收敛曲线见图 2。

3.4 准确度。 验证集 acc = **0.9412**、宏 F1 = **0.9414**，错误分类 65/1106（5.9%），训练 2.3 s；训练集宏 F1 = 0.9998，在训练集上几乎完全拟合。

3.5 损失函数与收敛曲线。 逻辑回归的损失是 softmax 交叉熵 + L2 正则，损失函数为 $-\sum_i \log P(y_i \mid x_i) + \frac{\|w\|^2}{2C}$。`experiment1.py` 在同一训练子集与同一 20k 词级 TF-IDF 表示上用 L-BFGS-B 优化该目标，逐次记录训练集上的平均交叉熵。

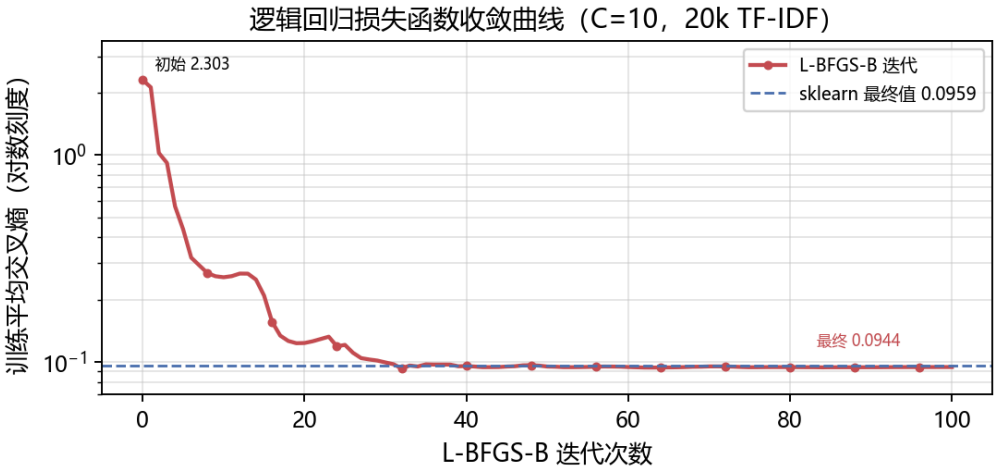


图 2（模块 B 的损失曲线）：训练平均交叉熵从 2.3026降到 0.0944，约 100 次迭代后基本稳定；scikit-learn 同配置的最终平均交叉熵为 0.0959，与曲线终点一致。损失曲线没有出现过拟合式反弹，说明 `C=10` 下 L-BFGS 能稳定收敛到较低的训练损失。

模块 B 小结：逻辑回归最优配置为 C = 10，验证集准确率 0.9412、宏 F1 0.9414；其损失收敛曲线平滑单调，概率输出在本次任务中是可靠的置信度估计。

4 两个算法的效果比较

比较项	模型 A：线性 SVM（LinearSVC）	模型 B：逻辑回归（LR）
验证集准确率 / 宏 F1	0.9430 / **0.9432**	0.9412 / 0.9414
错误分类数（/1106）	63（5.7%）	65（5.9%）
训练耗时（20k 表示，同一划分）	**0.2 s**	2.3 s
输出形式	决策函数得分（无概率）	**概率（可当作置信度）**
最优超参数	C = 1（C 增大后过拟合）	C = 10（C ⩾ 10 饱和）
5 折 × 3 种子宏 F1（稳健性）	**0.9444 ± 0.0025**	0.9390 ± 0.0018
平均残差（正确 / 错误分类样本）	0.186（0.11 / 1.44）	0.206（0.17 / 0.83）

残差统计上，SVM 的 63 个错误分类样本中有 58 个同样被 LR 错误分类，只有 5 例 SVM 独有、7 例 LR 独有；样本级残差的 Spearman 相关系数 ρ = 0.80，两者对各样本难易的判断高度一致。

比较结论:

1. **性能差距不显著** 宏 F1 相差 0.18 个百分点，而重复交叉验证给出的标准差约 0.002，在 1106 条验证样本的规模下无法判定两者孰优。
2. **两者效果相近** 92%（58/63）的错误分类样本被两个模型同时错误分类，样本级残差 ρ = 0.80，说明它们在该数据集上学到的几乎是同一条线性边界。
3. **选择模型建议** 更看重精度与训练成本时选线性 SVM（同精度下快约 10 倍，且重复交叉验证的方差更小）；需要概率输出、置信度、可解释性（权重可直接读作对数几率）或在线更新时选逻辑回归。
4. **共同瓶颈** 两者的残差都集中在短文本与语义相近的类别对上，这属于特征表示与数据本身的问题；RBF 核与集成模型的失败说明"换更强的模型"不是这个数据集的提升方向。

5 实验思考与扩展

Q1 为什么在这个数据集上，SVM和逻辑回归的表现通常很好？ TF-IDF 高维稀疏，类别信息集中在少量关键词上，样本分类近似于线性。

Q2 MLP模型的性能是否可以通过调整网络结构（增加层或神经元）或训练参数（增加epochs）来进一步提升？是否存在过拟合的风险？ 提升空间小且过拟合风险大。20k 输入接 256 隐藏单元已有约 512 万权重，而训练样本仅 6262 条；两个线性模型的训练集宏 F1 已达 0.9995 / 0.9998，说明训练集上已无欠拟合，继续加大容量只会记住噪声。要追平线性模型需同时增加 L2、早停或 dropout；更有效的方向是先替换特征表示。

Q3 除了TF-IDF，还有哪些文本表示方法？ ① 词袋 / n-gram,表示方法简单但会丢失权重；② Word2Vec / GloVe / FastText 适用于稠密低维的样本，可能抹平关键词判别性；③ ELMo / BERT / RoBERTa,上下文语义强，算力与数据需求大。

6 错误分析与改进

- **错误分析**：最终模型错误分类 62/1106（5.6%）；主要混淆为 7↔2、2→1、8→0；类别 F1 最低 0.85（类 2）；短文本（<100 词）准确率 0.891，长文本（>800 词）0.977。错误分类样本残差远高于正确样本（SVM 1.44 / 0.11，LR 0.83 / 0.17）。
- **清洗对照实验**：清洗越激进,准确度越差.原文的F1位0.9378,去除引用后降到0.9289,再去除邮件头后下降到0.9235,说明邮件头与引用本身携带话题信息，去噪声必须考虑实际情况。
- **研究发现**：① 使用词级与字符级 TF-IDF 联合清洗,会提升准确度:F1由0.9378 升到 0.9443；②SVM与LR两个模型对该数据集分类本质相同,集成二者没有明显收益。

7 实验结论

在固定划分与指标下，`LinearSVC(C=1)` 在 20k 词级 TF-IDF 上取得 acc 0.9430、宏 F1 0.9432；词级 + 字符级联合表示（60000 维）达到 acc 0.9439、宏 F1 **0.9443**。逻辑回归（C=10）仅低 0.18 个百分点，训练慢约 10 倍，但可输出概率置信度。两者残差高度相关（ρ = 0.80，92% 错误分类重叠），说明剩余误差来自特征表示与少数证据不足的样本，而非模型形式。最终选用SVM进行训练,提交文件 `predictions.csv` 。